OCOM5300M Artificial Intelligence Project Encoder-Cross Attention LSTM Decoder Fine-Tuning

In [1]:
```
# Custom Classes
from vocabulary import Vocabulary
from captions import Captions
from pipeline_concept import Encoder, Decoder, EncoderDecoderModel
from dataset_concept import ROCOv2_ImageCaptionDataset, ConceptVocabulary, ConceptLookup

# Deep Learning Functionality
import torch  # PyTorch library
import torch.nn as nn  # PyTorch neural network module
import torch.optim as optim  # optimization algorithms for training neural networks
from torch.utils.data import DataLoader  # creating custom data containers, batching, shuffling and data loading
capabilities

# Image Processing And Computer Vision Functionality
from torchvision import transforms  # image transformations, i.e., preprocessing, augmentations

# Data Handling And Analysis
import pandas as pd  # data manipulation and analysis using DataFrame structures

# Visualization
import matplotlib.pyplot as plt  # plotting and visualizing results

# Utilities
import os  # file and directory operations
```

In [2]:
```
# suppress Intel specific Math Kernel Library (MKL) warnings
os.environ['KMP_DUPLICATE_LIB_OK'] = 'True'
```

In [3]:
```
# CPU
device = torch.device('cpu')
```

In [4]:
```
# display the complete content of DataFrame columns without truncation
pd.set_option('display.max_colwidth', None);
```

In [5]:
```
# seed set to ensure reproducibility of results
seed = 42
```

In [6]:
```
# set PyTorch random seed
torch.manual_seed(seed)
```

Out[6]:
```
<torch._C.Generator at 0x23955b61870>
```

Captions

In [7]:
```
# training captions filename
training_captions_filename = "ROCOv2/train_captions.csv"
```

In [8]:
```
# instantiate training captions object
training_captions = Captions()
```

In [9]:
```
# load training captions
training_caption_data = training_captions.load_captions(training_captions_filename)
```

In [10]:
```
# clean training captions
training_captions.clean_captions();
```

In [11]:
```
# confirm number of cleaned training captions
len(training_caption_data)
```

Out[11]:
```
59958
```

In [12]:
```
# maximum caption length
maximum_caption_length = (training_captions.get_caption_length_percentile(99))
```

In [13]:
```
# validation captions filename
valid_captions_filename = "ROCOv2/valid_captions.csv"
```

In [14]:
```
# instantiate validation captions object
valid_captions = Captions()
```

In [15]:
```
# load validation captions
valid_caption_data = valid_captions.load_captions(valid_captions_filename)
```

In [16]:
```
# clean validation captions
valid_captions.clean_captions();
```

In [17]:
```
# confirm number of cleaned validation captions
len(valid_caption_data)
```

Out[17]:
```
9904
```

Vocabulary

In [18]:
```
# load  saved vocabulary
vocabulary = Vocabulary.load("ROCOv2/vocabulary.pt")
```

In [19]:
```
# confirm total number of tokens in the vocabulary
len(vocabulary)
```

Out[19]:
```
17602
```

In [20]:
```
# retrieve the index assigned to the padding token
pad_idx = vocabulary("<pad>")
```

Create Image Processing Pipeline

In [21]:
```
# image processing pipeline
transform = transforms.Compose([

    transforms.Resize(
        (224, 224), # 224 x 224
        antialias=True, # to prevent jagged edges, moiré patterns etc
    ),

    transforms.ToTensor(), # PyTorch neural network requires the image as a tensor of numerical values

    # Convert RGB tensor to BGR to match RadImageNet PyTorch preprocessing
    transforms.Lambda(lambda image: image[[2, 1, 0], :, :]),
```

```
    transforms.Normalize(
        mean=[0.5, 0.5, 0.5],
        std=[0.5, 0.5, 0.5],
    ),
])
```

Hyperparameters

In [22]:
```
# set hyperparameter, i.e., the dimensionality of the word embedding vectors in the model
embed_size = 256
```

In [23]:
```
# set hyperparameter, i.e., dimensionality of the hidden state in LSTM layers
hidden_size = 512
```

In [24]:
```
# set hyperparameter, i.e., the number of stacked recurrent layers in the neural network architecture
number_of_layers = 1
```

In [25]:
```
# set encoder feature dimension
encoder_dimension = 2048
```

In [26]:
```
# set attention dimension, i.e., the size of the intermediate feature space used to calculate attention scores
# for relevancy of each image region when the decoder is generating the next word
attention_dimension = 512
```

In [27]:
```
# pretrained RadImageNet ResNet-50 encoder weights
radimagenet_weights_path= "RadImageNet_pytorch/ResNet50.pt"
```

In [28]:
```
# specify the saved best-performing model file, i.e., based on the epoch with the lowest validation loss
selected_model = "cross_attention_lstm_pipeline_best.pth"
```

In [29]:
```
# learning rate for fine-tuning the pretrained encoder . If the learning rate is too large, fine-tuning can change
pretrained weights too aggressively
encoder_learning_rate = 1e-5
```

In [30]:
```
# learning rate for training the decoder
decoder_learning_rate = 1e-4
```

In [31]:
```
# size of penalty to be applied to large model weights to help reduce overfitting.
weight_decay = 1e-5
```

In [32]:
```
# maximum gradient size allowed before clipping to help prevent unstable weight updates
maximum_gradient_size = 1.0
```

In [33]:
```
#number of epochs
number_of_epochs = 10
```

In [34]:
```
# batch size
batch_size = 32
```

In [35]:
```
# early stopping patience
early_stopping_patience = 4
```

In [36]:
```
# learning rate scheduler patience
learning_rate_scheduler_patience = 2
```

In [37]:
```
# dropout probability: during training, 30% of selected activations are randomly set to zero to help reduce
overfitting
# This forces the model not to rely too heavily on particular neurons
# and can improve generalisation to unseen data to prevent overfitting
dropout = 0.3
```

In [38]:
```
# select visual gate type: scalar, vector or none
visual_gate_type = "none"
```

In [39]:
```
# numerical index assigned to the <pad> token in the vocabulary
pad_idx = vocabulary("<pad>")
```

In [40]:
```
# training image folder
training_image_folder="ROCOv2/train_images/train"
```

In [41]:
```
# validation image folder
validation_image_folder="ROCOv2/valid_images/valid"
```

In [42]:
```
# Factor used to reduce the learning rate when the scheduler is triggered
reduction_factor = 0.5
```

Load Both Checkpoints

In [43]:
```
# load the same training-only CUI vocabulary as decoder training
concept_vocabulary = ConceptVocabulary.load("lstm_pipeline_cui_vocabulary.json")
train_concepts = ConceptLookup("ROCOv2/pipeline_training.csv", concept_vocabulary, max_concepts=20)
valid_concepts = ConceptLookup("ROCOv2/pipeline_validation.csv", concept_vocabulary, max_concepts=20)
```

In [44]:
```
# load saved best decoder checkpoint
decoder_checkpoint = torch.load(selected_model, "cpu", weights_only=False)
```

In [45]:
```
# extract decoder configuration from the checkpoint
decoder_configuration = decoder_checkpoint["configuration"]
```

In [46]:
```
# store selected visual gate type in decoder configuration
decoder_configuration["visual_gate_type"] = visual_gate_type
```

In [47]:
```
# initialise encoder with frozen weights and batch normalisation using the pretrained RadImageNet ResNet-50 weights
encoder = Encoder(freeze_encoder=True, freeze_batch_norm=True, radimagenet_weights_path=radimagenet_weights_path)
```

In [48]:
```
# fine-tune only the selected later ResNet blocks.
encoder.fine_tune_backbone(train_layer4=True, train_layer3=False,train_layer2=False)
```

In [49]:
```
# initialise decoder
decoder = Decoder(
    vocab_size=decoder_configuration["vocab_size"],
    pad_idx=decoder_configuration["pad_idx"],
    encoder_dim=decoder_configuration["encoder_dim"],
    embed_size=decoder_configuration["embed_size"],
    hidden_size=decoder_configuration["hidden_size"],
    attention_dim=decoder_configuration["attention_dim"],
    num_layers=decoder_configuration["num_layers"],
    max_seq_length=maximum_caption_length,
    dropout=dropout,
    concept_vocab_size=decoder_configuration["concept_vocab_size"],
```

```
    visual_gate_type=visual_gate_type
)
```

In [50]:
```
# Load pretrained decoder weights from the saved checkpoint
decoder.load_state_dict(decoder_checkpoint["decoder"], strict=True)
```

Out[50]:
```
<All keys matched successfully>
```

Construct DataSets And Data Loaders

In [51]:
```
# create the training dataset using the training captions, images, vocabulary, and preprocessing transform
train_dataset = ROCOv2_ImageCaptionDataset(
    captions=training_caption_data,
    image_folder=training_image_folder,
    vocabulary=vocabulary,
    maximum_caption_length=maximum_caption_length,
    transform=transform,
    concept_lookup=train_concepts,
)
```

In [52]:
```
# create the validation dataset using the validation captions, images, vocabulary, and preprocessing transform
valid_dataset = ROCOv2_ImageCaptionDataset(
    captions=valid_caption_data,
    image_folder=validation_image_folder,
    vocabulary=vocabulary,
    maximum_caption_length=maximum_caption_length,
    transform=transform,
    concept_lookup=valid_concepts,
)
```

In [53]:
```
# create a dedicated random number generator for reproducible DataLoader shuffling
loader_generator = torch.Generator()
```

In [54]:
```
# seed the DataLoader generator
loader_generator.manual_seed(seed)
```

Out[54]:
```
<torch._C.Generator at 0x2390af9bb50>
```

In [55]:
```
# create training DataLoader
train_loader = DataLoader(
    train_dataset,
    batch_size=batch_size,
    shuffle=True,
    num_workers=0,
    generator=loader_generator,
)
```

In [56]:
```
# create validation DataLoader
valid_loader = DataLoader(
    valid_dataset,
    batch_size=batch_size,
    shuffle=False,
    num_workers=0,
)
```

Combined Encoder Decoder Model

In [57]:
```
# create encoder-decoder model
model = EncoderDecoderModel(encoder, decoder).to(device)
```

In [58]:
```
# retrieve first batch of images, captions, and caption lengths from the training DataLoader
images, captions, lengths, concept_ids, concept_probs = next(iter(train_loader))
```

In [59]:
```
# move the image batch to the selected computation device, i.e., CPU
images = images.to(device)
```

In [60]:
```
# move the caption batch to the selected computation device, i.e., CPU
captions = captions.to(device)
```

In [61]:
```
# move the lengths batch to the selected computation device, i.e., CPU
lengths = lengths.to(device).view(-1)
```

In [62]:
```
# define the token-level cross-entropy loss, ignoring padding tokens and summing loss over valid tokens
criterion = nn.CrossEntropyLoss(
    ignore_index=pad_idx,
```

```
    reduction="sum" # sum the loss across all valid caption tokens in the batch
)
```

In [63]:
```
# collect encoder parameters enabled for fine-tuning
encoder_parameters = [
    parameter
    for parameter in model.encoder.parameters()
    if parameter.requires_grad
]
```

In [64]:
```
# collect decoder parameters enabled for training
decoder_parameters = [
    parameter
    for parameter in model.decoder.parameters()
    if parameter.requires_grad
]
```

In [65]:
```
# configure the Adam optimizer with a lower learning rate for the encoder
# and a higher learning rate for the decoder during fine-tuning
optimizer = optim.Adam(
    [
        {
            "params": encoder_parameters,
            "lr": encoder_learning_rate,
        },
        {
            "params": decoder_parameters,
            "lr": decoder_learning_rate,
        },
    ],
    weight_decay=weight_decay,
)
```

In [66]:
```
# reduce the learning rate by half when validation loss stops improving for defined number of epochs
scheduler = optim.lr_scheduler.ReduceLROnPlateau(
    optimizer,
    mode="min", # treat a lower validation loss as an improvement
    factor=reduction_factor, # multiply the learning rate by 0.5 when reduction is triggered
    patience=learning_rate_scheduler_patience, # Wait defined epochs without improvement before reducing the
```

```
learning rate
)
```

In [67]:
```
# execute fine-tuning
fine_tuning_history = model.fine_tune_model(
    num_epochs=number_of_epochs,
    patience=early_stopping_patience,
    decoder_checkpoint=decoder_checkpoint,
    train_loader=train_loader,
    valid_loader=valid_loader,
    optimizer=optimizer,
    scheduler=scheduler,
    criterion=criterion,
    pad_idx=pad_idx,
    maximum_gradient_size=maximum_gradient_size,
    decoder_configuration=decoder_configuration,
    dropout=dropout,
    selected_model=selected_model,
    encoder_learning_rate=encoder_learning_rate,
    decoder_learning_rate=decoder_learning_rate,
    weight_decay=weight_decay,
)
```

```
Fine Tuning:   0%|          | 0/1874 [00:00<?, ?it/s]
Validation:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [1/10] - 12902.94s
Training loss: 3.7933 | Validation loss: 3.9078
Training perplexity: 44.4044 | Validation perplexity: 49.7885
Validation Loss Improved. Combined Checkpoint Saved To 'encoder_decoder_finetuned_best.pth'.
Fine Tuning:   0%|          | 0/1874 [00:00<?, ?it/s]
Validation:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [2/10] - 10465.84s
Training loss: 3.6875 | Validation loss: 3.8769
Training perplexity: 39.9439 | Validation perplexity: 48.2753
Validation Loss Improved. Combined Checkpoint Saved To 'encoder_decoder_finetuned_best.pth'.
Fine Tuning:   0%|          | 0/1874 [00:00<?, ?it/s]
Validation:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [3/10] - 10499.39s
Training loss: 3.6182 | Validation loss: 3.8649
Training perplexity: 37.2695 | Validation perplexity: 47.6982
Validation Loss Improved. Combined Checkpoint Saved To 'encoder_decoder_finetuned_best.pth'.
Fine Tuning:   0%|          | 0/1874 [00:00<?, ?it/s]
```

```
Validation:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [4/10] - 10438.92s
Training loss: 3.5591 | Validation loss: 3.8543
Training perplexity: 35.1313 | Validation perplexity: 47.1968
Validation Loss Improved. Combined Checkpoint Saved To 'encoder_decoder_finetuned_best.pth'.
Fine Tuning:   0%|          | 0/1874 [00:00<?, ?it/s]
Validation:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [5/10] - 10542.91s
Training loss: 3.5069 | Validation loss: 3.8477
Training perplexity: 33.3452 | Validation perplexity: 46.8838
Validation Loss Improved. Combined Checkpoint Saved To 'encoder_decoder_finetuned_best.pth'.
Fine Tuning:   0%|          | 0/1874 [00:00<?, ?it/s]
Validation:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [6/10] - 10554.14s
Training loss: 3.4582 | Validation loss: 3.8488
Training perplexity: 31.7610 | Validation perplexity: 46.9360
No Validation Improvement For 1 epoch(s).
Fine Tuning:   0%|          | 0/1874 [00:00<?, ?it/s]
Validation:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [7/10] - 10660.65s
Training loss: 3.4127 | Validation loss: 3.8519
Training perplexity: 30.3484 | Validation perplexity: 47.0816
No Validation Improvement For 2 epoch(s).
Fine Tuning:   0%|          | 0/1874 [00:00<?, ?it/s]
Validation:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [8/10] - 10785.87s
Training loss: 3.3701 | Validation loss: 3.8614
Training perplexity: 29.0805 | Validation perplexity: 47.5332
No Validation Improvement For 3 epoch(s).
Fine Tuning:   0%|          | 0/1874 [00:00<?, ?it/s]
Validation:   0%|          | 0/310 [00:00<?, ?it/s]
Epoch [9/10] - 10643.37s
Training loss: 3.3059 | Validation loss: 3.8728
Training perplexity: 27.2739 | Validation perplexity: 48.0775
No Validation Improvement For 4 epoch(s).
Early Stopping Triggered
```

Diagnostics

In [68]:
```
# display fine tuning history
fine_tuning_history
```

Out[68]:

	epoch	training loss	validation loss	training perplexity	validation perplexity	encoder learning rate	decoder learning rate	epoch seconds
0	1	3.793338	3.907784	44.404356	49.788506	0.000010	0.00010	12902.943903
1	2	3.687475	3.876919	39.943856	48.275253	0.000010	0.00010	10465.835465
2	3	3.618175	3.864894	37.269506	47.698204	0.000010	0.00010	10499.385091
3	4	3.559093	3.854325	35.131324	47.196751	0.000010	0.00010	10438.920588
4	5	3.506915	3.847673	33.345239	46.883822	0.000010	0.00010	10542.914494
5	6	3.458239	3.848786	31.760985	46.936047	0.000010	0.00010	10554.144196
6	7	3.412742	3.851882	30.348353	47.081577	0.000010	0.00010	10660.653411
7	8	3.370068	3.861428	29.080510	47.533168	0.000010	0.00010	10785.873804
8	9	3.305929	3.872814	27.273869	48.077485	0.000005	0.00005	10643.365873

In [69]:

```
def display_fine_tuning_history(fine_tuning_history) :

    # Create figure with a width of 10 inches and a height of 5 inches
    plt.figure(figsize=(10, 5))

    # label x-axis
    plt.xlabel("Epoch")

    # label y-axis
    plt.ylabel("Token loss")

    # title
    plt.title("End-To-End Encoder-Decoder Fine-Tuning")

    # display grid lines on the plot
    plt.grid(True)
    #plt.legend()
    #plt.show()

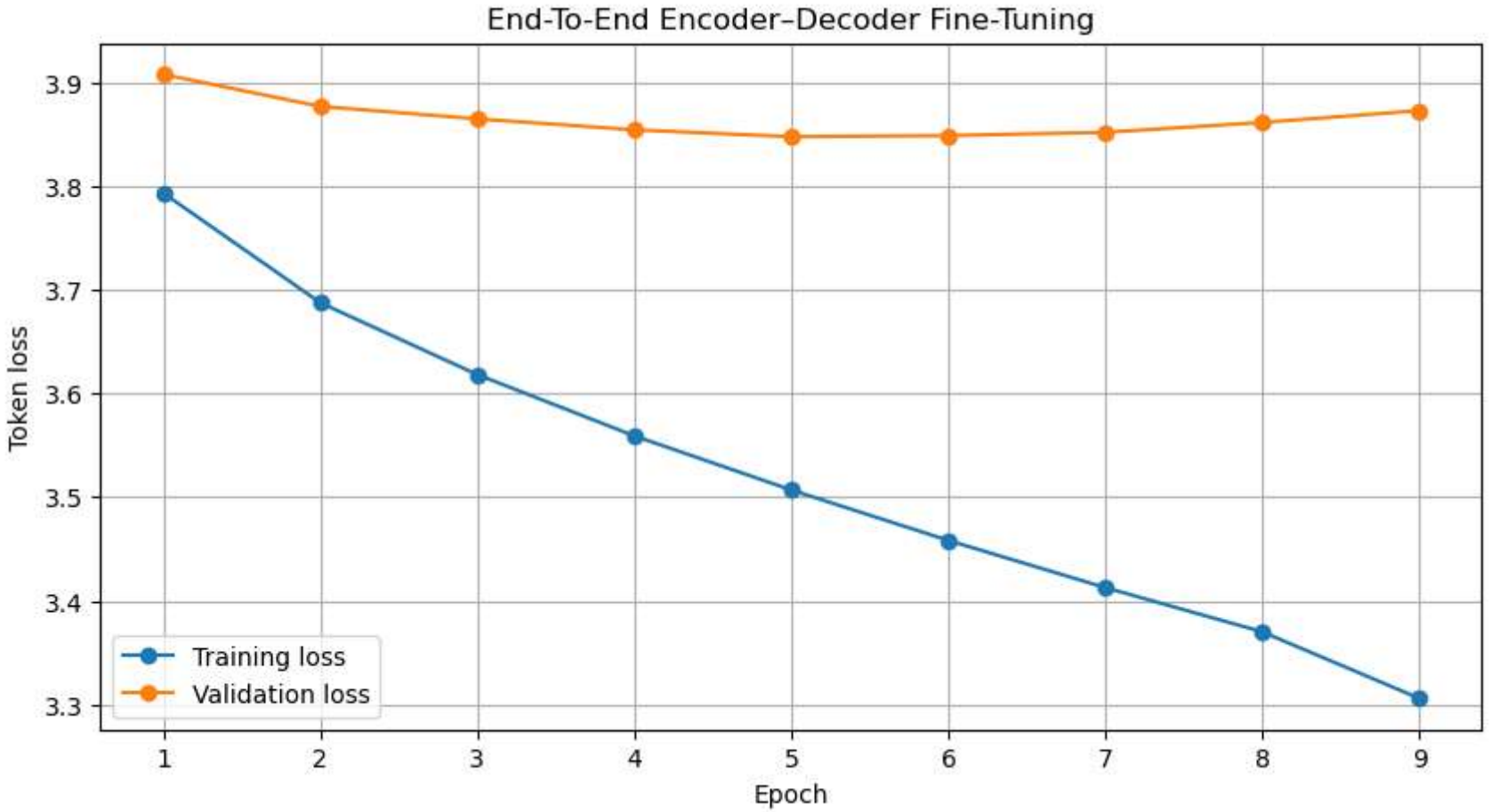
    # plot training loss for each epoch
    plt.plot(fine_tuning_history["epoch"], fine_tuning_history["training loss"], marker="o", label="Training loss")
```

```
    # # plot validation loss for each epoch
    plt.plot(fine_tuning_history["epoch"], fine_tuning_history["validation loss"], marker="o", label="Validation
loss")

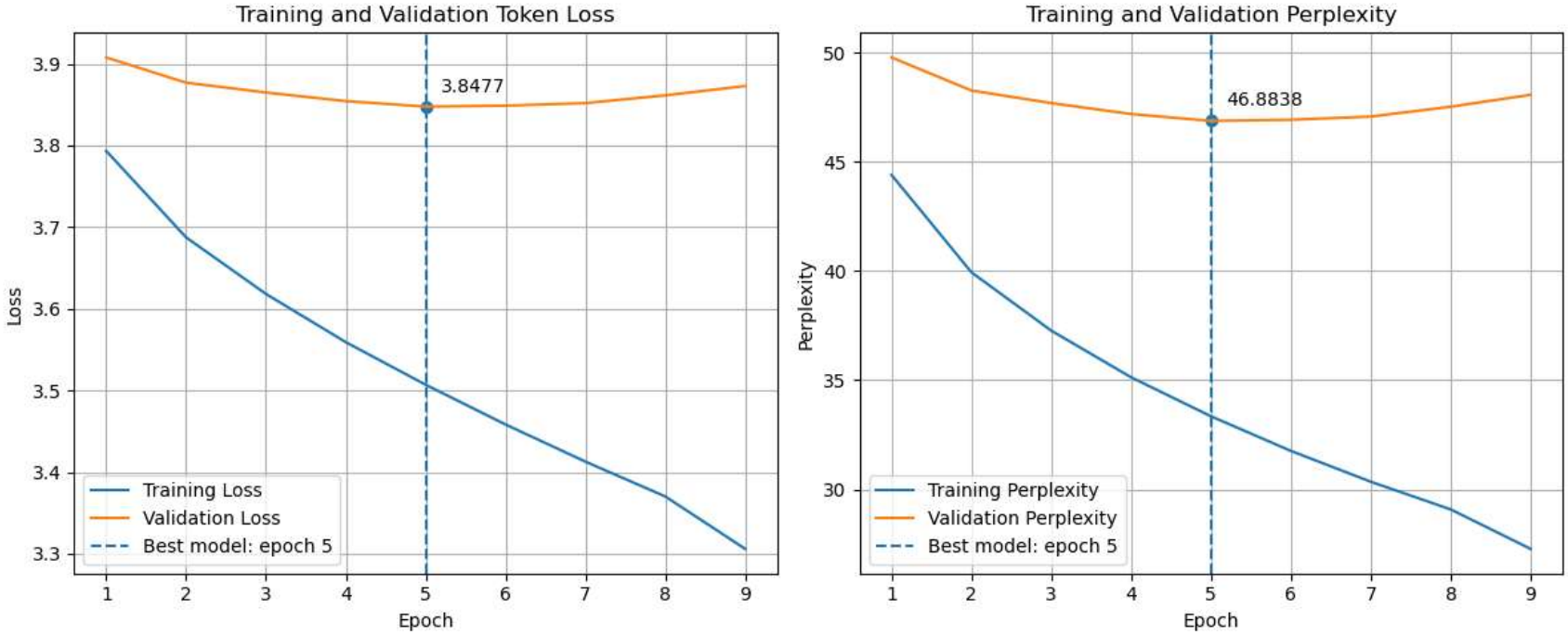
    #
    plt.legend()

    #
    plt.show()
```

In [70]:
```
display_fine_tuning_history(fine_tuning_history)
```



In [71]:
```
# display performance graphs
model.plot_model_performance(fine_tuning_history)
```



Out[71]:

```
EncoderDecoderModel(
  (encoder): Encoder(
    (cnn): Sequential(
      (0): Conv2d(3, 64, kernel_size=(7, 7), stride=(2, 2), padding=(3, 3), bias=False)
      (1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (2): ReLU(inplace=True)
      (3): MaxPool2d(kernel_size=3, stride=2, padding=1, dilation=1, ceil_mode=False)
      (4): Sequential(
        (0): Bottleneck(
          (conv1): Conv2d(64, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
          (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
          (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
          (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (relu): ReLU(inplace=True)
          (downsample): Sequential(
            (0): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
            (1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          )
        )
        (1): Bottleneck(
          (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
          (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
          (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
          (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (relu): ReLU(inplace=True)
        )
        (2): Bottleneck(
          (conv1): Conv2d(256, 64, kernel_size=(1, 1), stride=(1, 1), bias=False)
          (bn1): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (conv2): Conv2d(64, 64, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
          (bn2): BatchNorm2d(64, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (conv3): Conv2d(64, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
          (bn3): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
          (relu): ReLU(inplace=True)
        )
      )
      (5): Sequential(
        (0): Bottleneck(
```

```
      (conv1): Conv2d(256, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(256, 512, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (3): Bottleneck(
      (conv1): Conv2d(512, 128, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(128, 128, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(128, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(128, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (6): Sequential(
    (0): Bottleneck(
```

```
      (conv1): Conv2d(512, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(512, 1024, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (3): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (4): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
```

```
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (5): Bottleneck(
      (conv1): Conv2d(1024, 256, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(256, 256, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(256, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(256, 1024, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(1024, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
  )
  (7): Sequential(
    (0): Bottleneck(
      (conv1): Conv2d(1024, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(2, 2), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
      (downsample): Sequential(
        (0): Conv2d(1024, 2048, kernel_size=(1, 1), stride=(2, 2), bias=False)
        (1): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      )
    )
    (1): Bottleneck(
      (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
      (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
      (relu): ReLU(inplace=True)
    )
    (2): Bottleneck(
      (conv1): Conv2d(2048, 512, kernel_size=(1, 1), stride=(1, 1), bias=False)
      (bn1): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
```

```
        (conv2): Conv2d(512, 512, kernel_size=(3, 3), stride=(1, 1), padding=(1, 1), bias=False)
        (bn2): BatchNorm2d(512, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (conv3): Conv2d(512, 2048, kernel_size=(1, 1), stride=(1, 1), bias=False)
        (bn3): BatchNorm2d(2048, eps=1e-05, momentum=0.1, affine=True, track_running_stats=True)
        (relu): ReLU(inplace=True)
      )
    )
  )
 )
 (decoder): Decoder(
   (concept_embeddings): Embedding(1810, 256, padding_idx=0)
   (concept_projection): Linear(in_features=256, out_features=2048, bias=False)
   (embedding): Embedding(17602, 256, padding_idx=0)
   (cross_attention): AdditiveCrossAttention(
     (visual_projection): Linear(in_features=2048, out_features=512, bias=True)
     (hidden_projection): Linear(in_features=512, out_features=512, bias=True)
     (attention_score): Linear(in_features=512, out_features=1, bias=False)
   )
   (lstm_cells): ModuleList(
     (0): LSTMCell(2304, 512)
   )
   (initial_hidden): ModuleList(
     (0): Linear(in_features=2048, out_features=512, bias=True)
   )
   (initial_cell): ModuleList(
     (0): Linear(in_features=2048, out_features=512, bias=True)
   )
   (visual_gate): Linear(in_features=512, out_features=1, bias=True)
   (dropout): Dropout(p=0.3, inplace=False)
   (hidden_to_embed): Linear(in_features=512, out_features=256, bias=True)
   (output_layer): Linear(in_features=256, out_features=17602, bias=False)
 )
)
```

In [72]:

```python
def plot_gate_diagnostics(model, data_loader, device):
    """
    Calculate and plot the mean visual gate value at each caption position.

    Args:
        model : trained encoder decoder model
        data_loader: validation DataLoader
        device: device used for model calculations
```

```
    Returns:
        pd.DataFrame: gate diagnostics by caption timestep
    """

    try:

        # calculate mean visual gate values by caption position
        gate_diagnostics = model.calculate_gate_mean_by_timestep(
            data_loader,
            device
        )

        # create figure
        plt.figure(figsize=(10, 5))

        # plot mean visual gate by caption position
        plt.plot(
            gate_diagnostics["Timestep"],
            gate_diagnostics["Mean Gate"],
            marker="o"
        )

        # configure plot
        plt.xlabel("Caption Position")
        plt.ylabel("Mean Visual Gate")
        plt.title("Mean Visual Gate by Caption Position")
        plt.ylim(0, 1)
        plt.grid(True)

        # display plot
        plt.show()

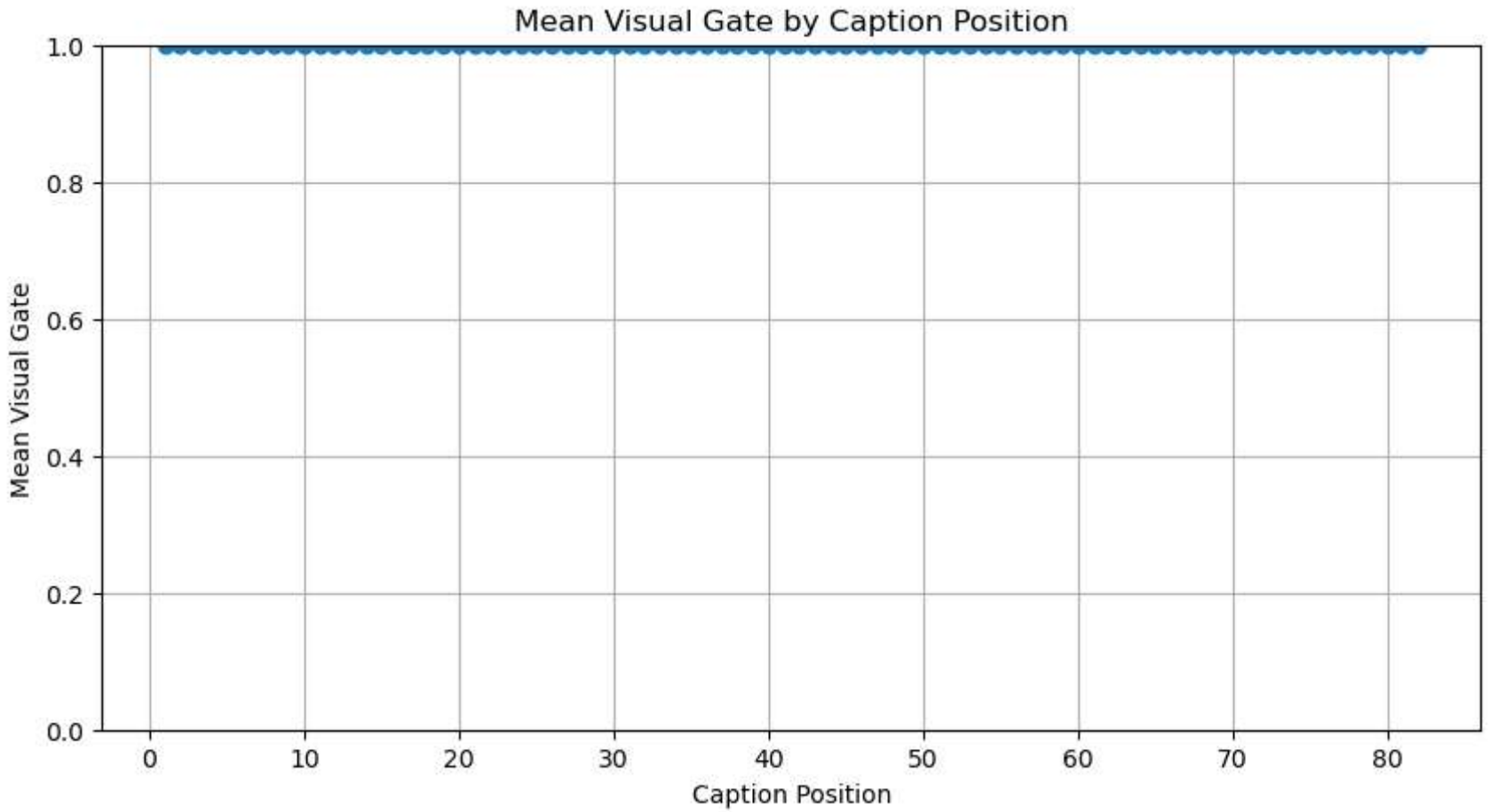
        return gate_diagnostics

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [73]:
```
# the gate is a learned control value that regulates how much visual information from the image is allowed into the decoder at each caption timestep.
```

```
# calculate mean visual gate value by caption timestep
gate_diagnostics = plot_gate_diagnostics(model, valid_loader, device)
```

```
Gate diagnostics:   0%|          | 0/310 [00:00<?, ?it/s]
```



In [74]:
```
# display gate diagnostics
display(gate_diagnostics)
```

	Timestep	Mean Gate	Valid Captions
0	1	1.0	9904.0
1	2	1.0	9903.0
2	3	1.0	9898.0
3	4	1.0	9826.0
4	5	1.0	9715.0
...	...	...	...
77	78	1.0	186.0
78	79	1.0	177.0
79	80	1.0	170.0
80	81	1.0	162.0
81	82	1.0	152.0

82 rows × 3 columns

In [75]:
```python
def plot_visual_context_diagnostics(model, data_loader, device):
    """
    Calculate and plot the mean visual context magnitude
    at each caption position.

    Args:
        model : trained encoder decoder model
        data_loader: validation DataLoader
        device: device used for model calculations

    Returns:
        pd.DataFrame: visual context diagnostics by caption timestep
    """

    try:

        # calculate mean visual context magnitude by caption position
```

```
        visual_context_diagnostics = (
            model.calculate_visual_context_magnitude_by_timestep(
                data_loader,
                device
            )
        )

        # create figure
        plt.figure(figsize=(10, 5))

        # plot mean visual context magnitude by caption position
        plt.plot(
            visual_context_diagnostics["Timestep"],
            visual_context_diagnostics["Mean Visual Context Magnitude"],
            marker="o"
        )

        # configure plot
        plt.xlabel("Caption Position")
        plt.ylabel("Mean Visual Context Magnitude")
        plt.title("Mean Visual Context Magnitude by Caption Position")
        plt.grid(True)

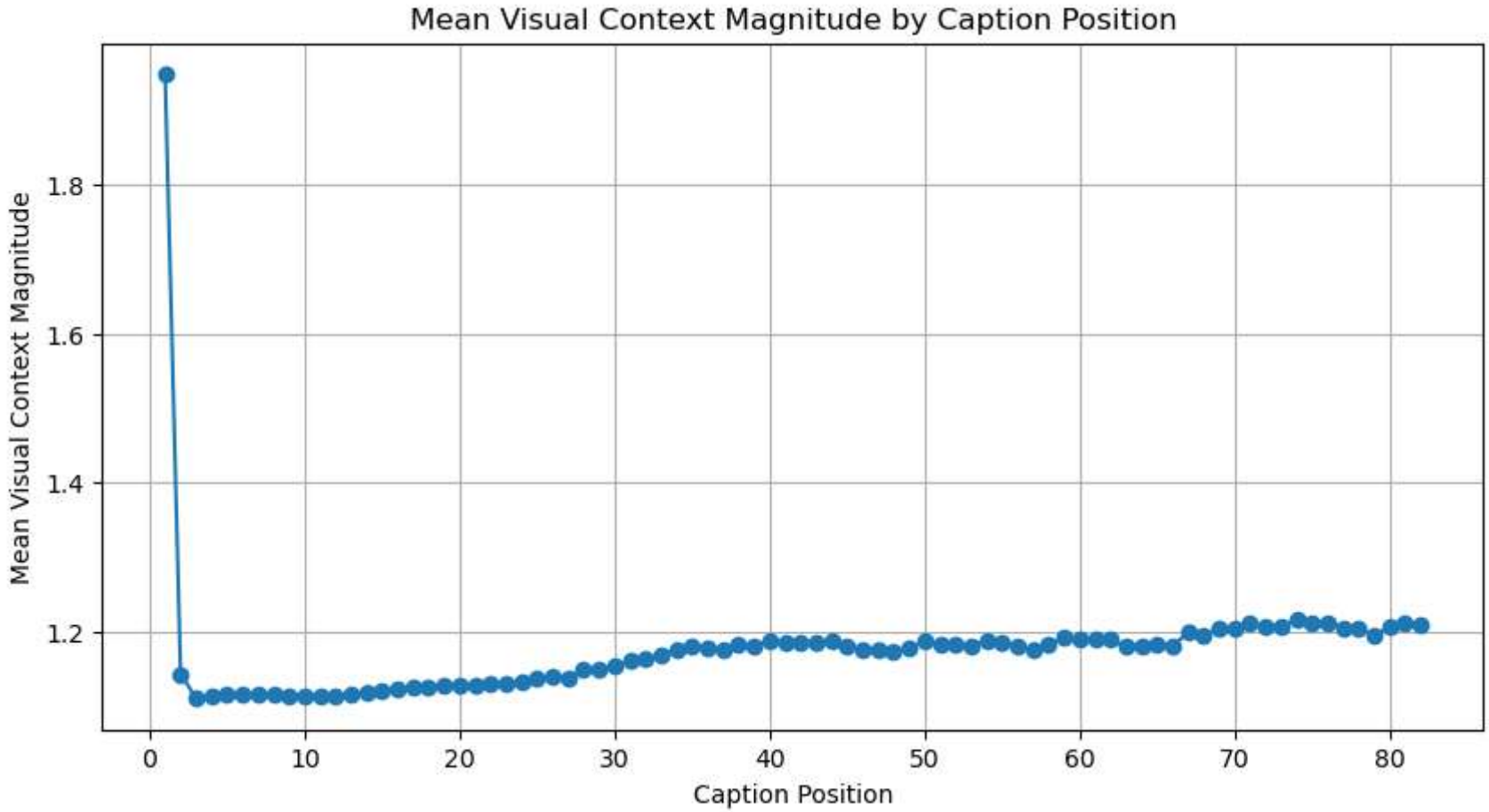
        # display plot
        plt.show()

        return visual_context_diagnostics

    except Exception as e:
            raise RuntimeError(f"Exception : {str(e)}")
```

In [76]:
```
# visual context magnitude
visual_context_diagnostics = plot_visual_context_diagnostics(model, valid_loader, device)
```

```
Visual Context Diagnostics:   0%|          | 0/310 [00:00<?, ?it/s]
```



In [77]:
```
# display visual context magnitude
display(visual_context_diagnostics)
```

	Timestep	Mean Visual Context Magnitude	Valid Captions
0	1	1.948860	9904.0
1	2	1.143103	9903.0
2	3	1.110347	9898.0
3	4	1.112224	9826.0
4	5	1.114563	9715.0
...	...	...	...
77	78	1.204490	186.0
78	79	1.194386	177.0
79	80	1.207202	170.0
80	81	1.211922	162.0
81	82	1.209616	152.0

82 rows × 3 columns

In [78]:
```python
def plot_attention_entropy_diagnostics(model, data_loader, device):
    """
    Calculate and plot normalized attention entropy
    at each caption position.

    Args:
        model : trained encoder decoder model
        data_loader: validation DataLoader
        device: device used for model calculations

    Returns:
        pd.DataFrame: attention entropy diagnostics
        by caption timestep
    """
    try:

        # calculate attention entropy by caption position
```

```
        attention_entropy_diagnostics = (model.calculate_attention_entropy_by_timestep(data_loader, device))

        # create figure
        plt.figure(figsize=(10, 5))

        # plot normalized attention entropy
        plt.plot(
            attention_entropy_diagnostics["Timestep"],
            attention_entropy_diagnostics[
                "Normalized Attention Entropy"
            ],
            marker="o"
        )

        # configure plot
        plt.xlabel("Caption Position")
        plt.ylabel("Normalized Attention Entropy")
        plt.title("Attention Entropy by Caption Position")

        # normalized entropy ranges from 0 to 1
        plt.ylim(0, 1)
        plt.grid(True)

        # display plot
        plt.show()

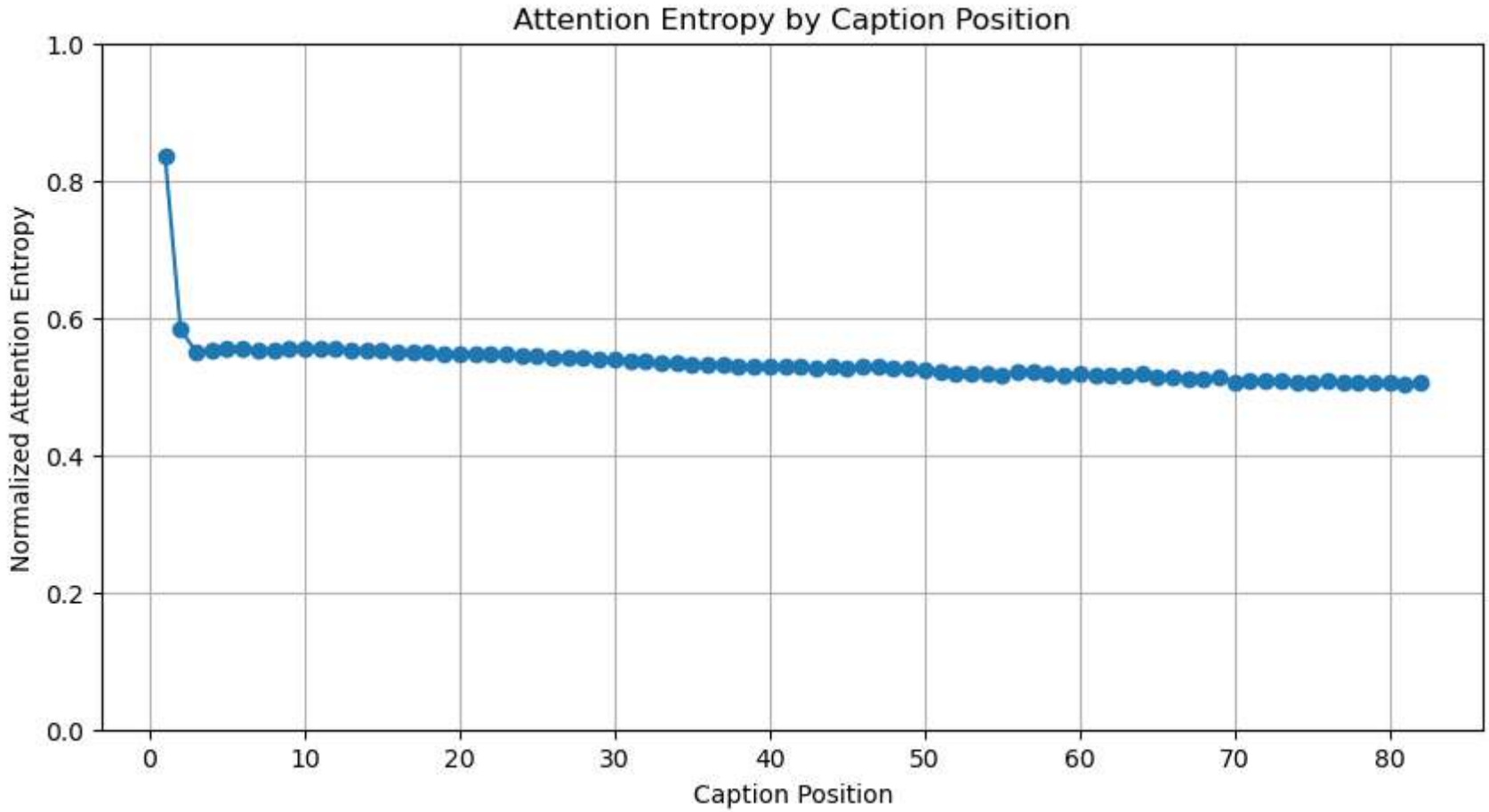
        return attention_entropy_diagnostics

    except Exception as e:
            raise RuntimeError(f"Exception : {str(e)}")
```

In [79]:

```
# attention entropy shows how concentrated or spread out the model's visual attention is across the image regions
at each caption position
# attention entropy by timestep
attention_entropy_diagnostics = plot_attention_entropy_diagnostics(model, valid_loader, device)
```

```
Attention entropy diagnostics:   0%|          | 0/310 [00:00<?, ?it/s]
```



In [80]:
```
# display attention entropy diagnostics
display(attention_entropy_diagnostics)
```

	Timestep	Mean Attention Entropy	Normalized Attention Entropy	Valid Captions
0	1	3.252246	0.835662	9904.0
1	2	2.268321	0.582843	9903.0
2	3	2.141096	0.550153	9898.0
3	4	2.148363	0.552020	9826.0
4	5	2.156541	0.554122	9715.0
...	...	...	...	...
77	78	1.972061	0.506719	186.0
78	79	1.967953	0.505664	177.0
79	80	1.966647	0.505328	170.0
80	81	1.958716	0.503291	162.0
81	82	1.964204	0.504701	152.0

82 rows × 4 columns

In [81]:
```
# calculate the out-of-vocabulary statistics for the validation captions against the final vocabulary
valid_oov = vocabulary.calculate_oov_statistics(valid_caption_data, dataset_name="Validation")
```

In [82]:
```
def plot_oov_statistics(oov_statistics):
    """
    Display token-level and caption-level out-of-vocabulary (OOV) statistics
    as horizontal percentage-based stacked bar charts with summary tables

    Args:
        oov_statistics (dict): OOV statistics returned by calculate_oov_statistics()
    """
    try:

        # retrieve dataset name
        dataset_name = oov_statistics["dataset"]
```

```
        # retrieve total number of tokens
        total_tokens = oov_statistics["total_tokens"]

        # retrieve total number of out-of-vocabulary tokens
        oov_tokens = oov_statistics["oov_tokens"]

        # retrieve precentage of tokens out-of-vocabulary
        token_oov_percentage = oov_statistics["oov_rate"]

        # calculate percentage of tokens that are present in the vocabulary
        in_vocabulary_percentage = 100 - token_oov_percentage

        # retrieve total number of captions
        total_captions = oov_statistics["total_captions"]

        # retrieve number of captions containing at least one out-of-vocabulary token
        captions_with_oov = oov_statistics["captions_with_oov"]

        # retrieve percentage of captions containing at least one out-of-vocabulary token
        caption_oov_percentage = oov_statistics["oov_caption_percentage"]

        # calculate percentage of captions containing no OOV words
        captions_without_oov_percentage = 100 - caption_oov_percentage

        # create vertically stacked charts
        fig, axes = plt.subplots(2, 1, figsize=(10, 5))

        # plot the percentage of tokens that are present in the vocabulary
        axes[0].barh(0, in_vocabulary_percentage, height=0.25, color="black", edgecolor="black", linewidth=0.8)

        # plot the percentage of out-of-vocabulary tokens
        axes[0].barh(0, token_oov_percentage, left=in_vocabulary_percentage, height=0.25, color="lightgrey",
edgecolor="grey", linewidth=0.8, hatch="//")

        # set token-level chart title
        axes[0].set_title(f'{dataset_name} Token-Level OOV', loc="left", pad=10)

        # display token category labels centred above each bar segment
        axes[0].text(in_vocabulary_percentage / 2, 0.22, "In Vocabulary", ha="center", va="bottom")
        axes[0].text(in_vocabulary_percentage + (token_oov_percentage / 2), 0.22, "OOV", ha="center", va="bottom")

        # create token-level summary table
```

```
        token_table = axes[0].table(cellText=[[f"{total_tokens:,}", f"{oov_tokens:,}", f"
{token_oov_percentage:.2f}%"]],
                                    colLabels=["Total Tokens", "OOV Tokens", "OOV Rate (%)"], cellLoc="center",
loc="bottom",
                                    bbox=[0, -0.42, 1, 0.30])

        # set token-level table borders
        for cell in token_table.get_celld().values():
            cell.set_linewidth(0.8)

        # plot the percentage of captions containing no OOV words
        axes[1].barh(0, captions_without_oov_percentage, height=0.25, color="black", edgecolor="black",
linewidth=0.8)

        # plot the percentage of captions containing at least one OOV word
        axes[1].barh(0, caption_oov_percentage, left=captions_without_oov_percentage, height=0.25,
                     color="lightgrey", edgecolor="grey", linewidth=0.8, hatch="//")

        # set caption-level chart title
        axes[1].set_title(f'{dataset_name} Caption-Level OOV', loc="left", pad=10)

        # display caption category labels centred above each bar segment
        axes[1].text(captions_without_oov_percentage / 2, 0.22, "Without OOV", ha="center", va="bottom")
        axes[1].text(captions_without_oov_percentage + (caption_oov_percentage / 2), 0.22, "With OOV", ha="center",
va="bottom")

        # create caption-level summary table
        caption_table = axes[1].table(cellText=[[f"{total_captions:,}", f"{captions_with_oov:,}", f"
{caption_oov_percentage:.2f}%"]],
                                      colLabels=["Total Captions", "Captions With OOV Word", "OOV Caption (%)"],
cellLoc="center", loc="bottom",
                                      bbox=[0, -0.42, 1, 0.30])

        # set caption-level table borders
        for cell in caption_table.get_celld().values():
            cell.set_linewidth(0.8)

        # remove axes, tick marks and borders
        for axis in axes:

            # set percentage scale
            axis.set_xlim(0, 100)
```

```
            # set vertical spacing
            axis.set_ylim(-0.45, 0.55)

            # remove axis tick marks
            axis.set_xticks([])
            axis.set_yticks([])

            # remove chart borders
            for spine in axis.spines.values():
                spine.set_visible(False)

        # reduce spacing between charts
        plt.subplots_adjust(hspace=0.75)

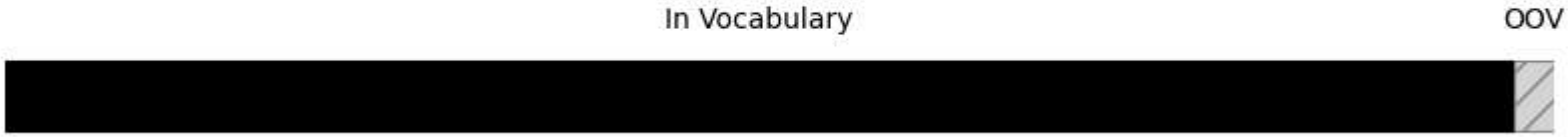
        # display charts
        plt.show()

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [83]:

```
# display OOV statistics graphically
plot_oov_statistics(valid_oov)
```

Validation Token-Level OOV



Total Tokens	OOV Tokens	OOV Rate (%)
227,293	5,825	2.56%

Validation Caption-Level OOV



Total Captions	Captions With OOV Word	OOV Caption (%)
9,904	3,082	31.12%

In [84]:
```
def plot_caption_length_comparison(training_captions, valid_captions):
    """
    Calculate, summarise and plot training and validation caption-length statistics

    Args:
        training_captions: cleaned training Captions object
        valid_captions: cleaned validation Captions object

    Returns:
        tuple:
            training_caption_lengths: training caption lengths
            validation_caption_lengths: validation caption lengths
```

```
        caption_length_summary: summary statistics for training and validation caption lengths
    """

    try:

        # calculate training caption lengths
        training_caption_lengths = (
            training_captions.calculate_caption_lengths()
        )

        # calculate validation caption lengths
        validation_caption_lengths = (
            valid_captions.calculate_caption_lengths()
        )

        # summarise training and validation caption lengths
        caption_length_summary = pd.DataFrame([
            {
                "Dataset": "Training",
                "Captions": len(training_caption_lengths),
                "Mean Length": training_caption_lengths.mean(),
                "Median Length": training_caption_lengths.median(),
                "Minimum Length": training_caption_lengths.min(),
                "Maximum Length": training_caption_lengths.max(),
                "Std Dev": training_caption_lengths.std(),
            },
            {
                "Dataset": "Validation",
                "Captions": len(validation_caption_lengths),
                "Mean Length": validation_caption_lengths.mean(),
                "Median Length": validation_caption_lengths.median(),
                "Minimum Length": validation_caption_lengths.min(),
                "Maximum Length": validation_caption_lengths.max(),
                "Std Dev": validation_caption_lengths.std(),
            }
        ])

        # select caption-length statistics for graphical comparison
        length_statistics = (
            caption_length_summary
            .set_index("Dataset")[
                [
```

```
                    "Mean Length",
                    "Median Length",
                    "Minimum Length",
                    "Maximum Length",
                    "Std Dev"
                ]
            ]
        )

        # create grouped bar chart
        length_statistics.T.plot(
            kind="bar",
            figsize=(10, 5)
        )

        # configure plot
        plt.xlabel("Caption Length Statistic")
        plt.ylabel("Caption Length (words)")
        plt.title("Training and Validation Caption Length Comparison")
        plt.xticks(rotation=45)
        plt.legend(title="Dataset")
        plt.grid(axis="y")

        # prevent labels from overlapping
        plt.tight_layout()

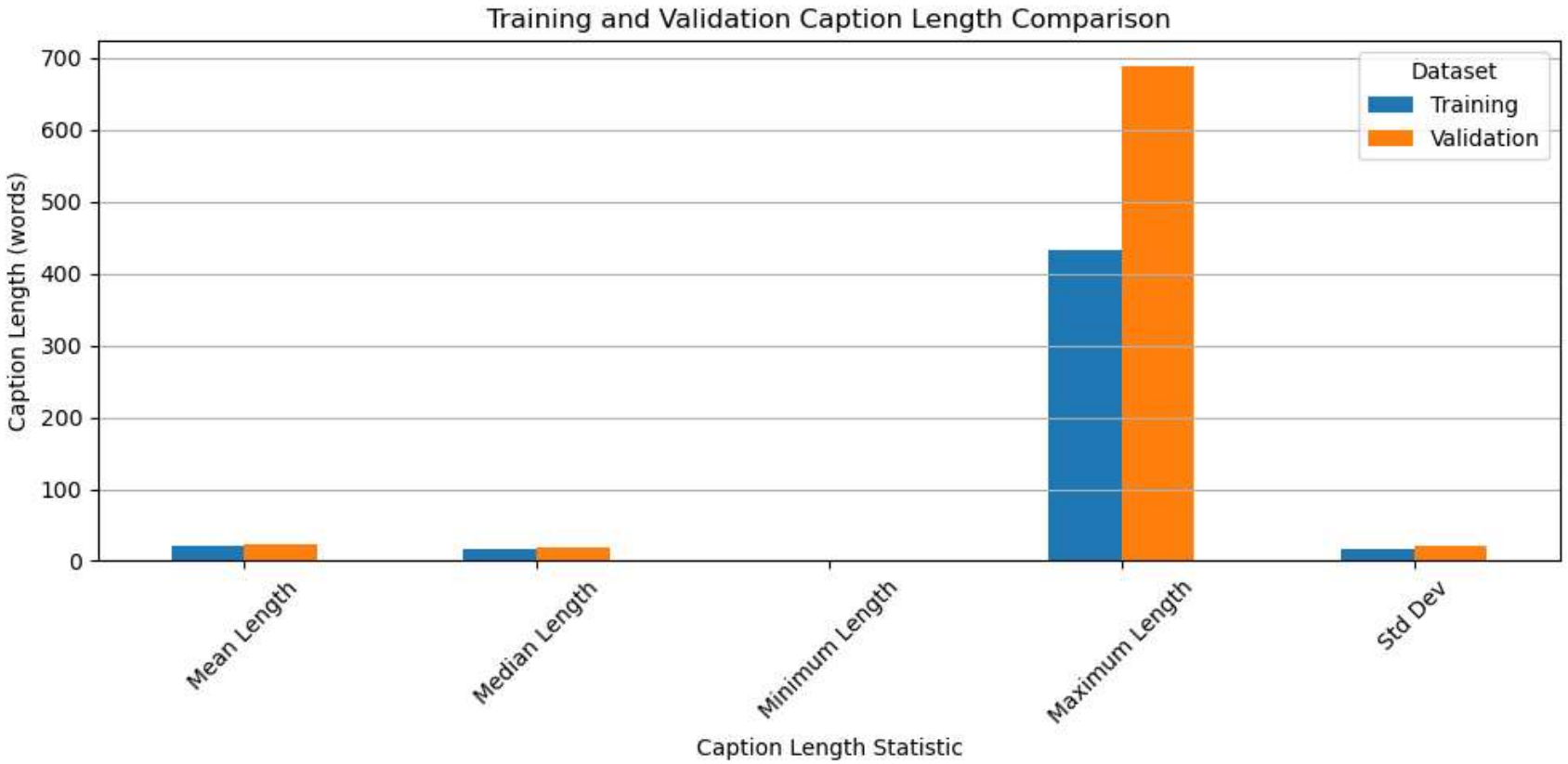
        # display plot
        plt.show()

        return (
            training_caption_lengths,
            validation_caption_lengths,
            caption_length_summary
        )

    except Exception as e:
        raise RuntimeError(f"Exception : {str(e)}")
```

In [85]:

```
# calculate, summarise and compare training and validation caption lengths
training_caption_lengths, validation_caption_lengths, caption_length_summary =
plot_caption_length_comparison(training_captions, valid_captions)
```



In [86]:
```
# display caption length summary
display(caption_length_summary)
```

	Dataset	Captions	Mean Length	Median Length	Minimum Length	Maximum Length	Std Dev
0	Training	59958	21.378615	17.0	1	432	16.277227
1	Validation	9904	22.949616	18.0	0	689	21.833881

In [87]:
```
def plot_caption_length_loss(model, data_loader, device, bin_size=10):
    """
    Calculate and plot mean validation token loss

    Args:
        model : trained encoder decoder model
```

```
        data_loader: validation DataLoader
        device: device used for model calculations
        bin_size (int): number of words in each caption-length range

    Returns:
        tuple:
            validation_caption_losses: individual caption-level loss results
            caption_length_loss_summary: mean loss grouped by caption-length range
    """

    try:

        # calculate individual caption losses and grouped summary
        validation_caption_losses, caption_length_loss_summary = (
            model.calculate_loss_by_caption_length(data_loader, device, bin_size=bin_size))

        # create figure
        plt.figure(figsize=(10, 5))

        # plot mean token loss by caption-length range
        plt.plot(
            caption_length_loss_summary["Length Range"].astype(str),
            caption_length_loss_summary["Mean_Token_Loss"],
            marker="o"
        )

        # configure plot
        plt.xlabel("Caption Length (words)")
        plt.ylabel("Mean Token Loss")
        plt.title("Validation Loss by Caption Length")
        plt.xticks(rotation=45)
        plt.grid(True)

        # prevent labels from overlapping
        plt.tight_layout()

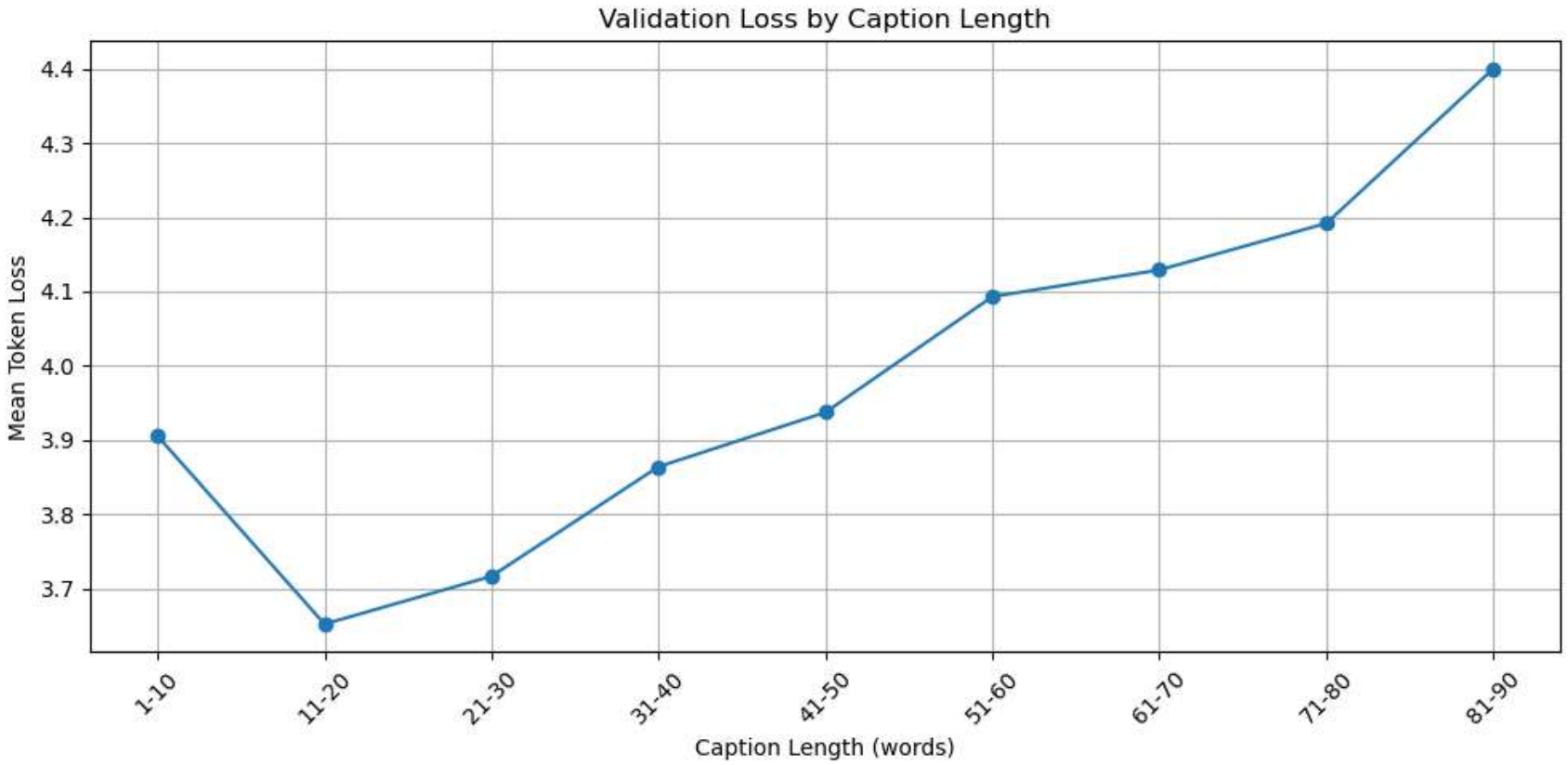
        # display plot
        plt.show()

        return (validation_caption_losses, caption_length_loss_summary)
```

```
    except Exception as e:
            raise RuntimeError(f"Exception : {str(e)}")
```

In [88]:
```
# calculate individual validation-caption loss statistics
validation_caption_losses, caption_length_loss_summary = (plot_caption_length_loss(model, valid_loader, device,
bin_size=10))
```

```
Loss By Caption Length:   0%|          | 0/310 [00:00<?, ?it/s]
```



In [89]:
```
# display vaidation caption losses
display(validation_caption_losses)
```

	Caption Length	Mean Token Loss	Total Caption Loss	Length Range
0	17	2.100046	37.800823	11-20
1	45	3.745504	172.293198	41-50
2	81	5.146397	422.004547	81-90
3	6	3.413618	23.895325	1-10
4	4	4.092292	20.461458	1-10
...	...	...	...	...
9899	22	3.631970	83.535309	21-30
9900	23	2.187250	52.494007	21-30
9901	44	3.249363	146.221344	41-50
9902	6	6.871994	48.103954	1-10
9903	80	4.826284	390.929047	71-80

9904 rows × 4 columns

In [90]:
```
# display caption lenght loss summary
display(caption_length_loss_summary)
```

	Length Range	Captions	Mean_Caption_Length	Mean_Token_Loss	Mean_Total_Caption_Loss
0	1-10	2115	7.134279	3.905405	31.443929
1	11-20	3570	15.245938	3.652103	59.271182
2	21-30	2077	24.937891	3.716547	96.565770
3	31-40	1022	35.006849	3.864006	139.222917
4	41-50	490	45.087755	3.937329	181.536340
5	51-60	259	55.355212	4.093145	230.707664
6	61-70	139	65.366906	4.129067	274.187871
7	71-80	80	75.487500	4.191866	320.794245
8	81-90	152	81.000000	4.399941	360.795155

References

Cruz Diaz, N.P. and Mana Lopez, M, 'An analysis of biomedical tokenization: problems and strategies', Proceedings of the Sixth International Workshop on Health Text Mining and Information Analysis', pages 40-49 (2015). Available At: https://aclanthology.org/W15-2605/

He, B., Guan, Y., Cheng, J., Cen, K. and Hua, W. CRFs Based De-identification Of Medical Records. Science Direct, 2015. Available at: https://doi.org/10.1016/j.jbi.2015.08.012

Moon, S.R., Pakhomov, S., Ryan, J. and Melton, G.B., 'Automated non-alphanumeric symbol resolution in clinical texts', AMIA Annual Symposium Proceedings (2011) pages 979–986. Available At : https://pmc.ncbi.nlm.nih.gov/articles/PMC3243158/

OCOM5100M Programming For Data Science

OCOM5101M Data Science

OCOM5200M Machine Learning

OCOM5203M Deep Learning

OCOM5203M Deep Learning Module, Summative Assessment 2

OCOM5204M Data Mining And Analytics

Project Jupyter, Jupyter Notebook documentation. Available At: https://jupyter-notebook.readthedocs.io/en/stable/

Python Software Foundation, Python 3.13 documentation (2026). Available At: https://docs.python.org/3.13/